Passaggio per valore e pilotaggio del 74HC595

Lezione 2 — 2MAK, SST-INF-01c — Funzioni, memoria, `#define` e il primo programma per il 595

22/10/2026

Nota

In questa lezione riprendiamo le **funzioni con parametri** della dispensa A01 e vediamo **che cosa succede in memoria** quando una funzione riceve un numero: il **passaggio per valore**. Poi passiamo alla pratica: montiamo su **Tinkercad** il circuito del **74HC595** con 8 LED (lo stesso della dispensa A04 di elettronica) e scriviamo, passo dopo passo, il programma che lo **pilota**, fino a ridurlo a un **ciclo `for`** di poche righe.

1 Ripresa: la funzione `lampeggio`

Abbiamo detto che una **funzione** è come una **macro**: una **raccolta di istruzioni** raggruppate sotto un **singolo nome**. Quando chiamo `lampeggio`, Arduino esegue tutte le istruzioni che ci sono dentro. L'ultima volta avevamo scritto una funzione con tre **parametri**: il **piedino** da far lampeggiare, il **numero di lampeggi** e il **tempo** di ciascuna fase (cioè la velocità del lampeggio).

```
void lampeggio(int pin, int n, int t)
{
  for (int i = 0; i < n; i++)
  {
    digitalWrite(pin, HIGH);
    delay(t);
    digitalWrite(pin, LOW);
    delay(t);
  }
}

void loop()
{
  // piedino 5: 3 lampeggi da 200 ms
  lampeggio(5, 3, 200);
  // piedino 12: 7 lampeggi da 500 ms
  lampeggio(12, 7, 500);
}
```

La funzione **si adatta** ai parametri che le passiamo: senza riscrivere ogni volta il codice, chiamo tutti i lampeggi che voglio dicendo **quale piedino**, **quante volte** e **con che tempo**. Questo esempio sta “sotto il cappello” delle **funzioni con passaggio per valore**.

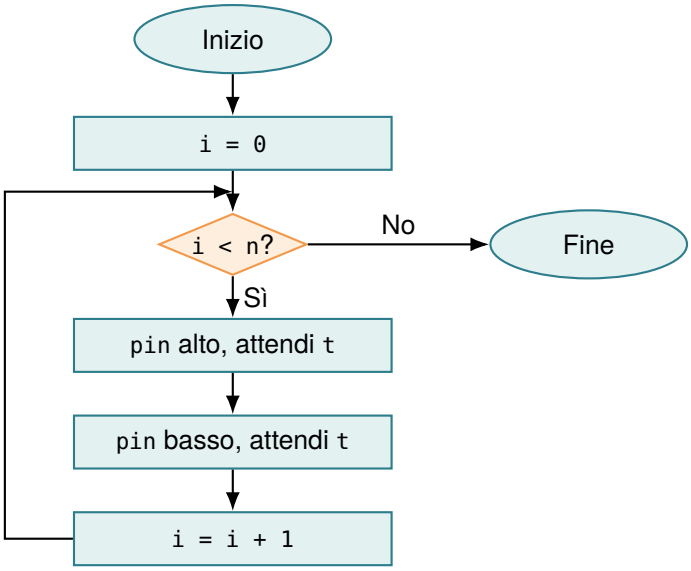


Figura 1: Diagramma di flusso della funzione `lampeggio(pin, n, t)`.

2 Che cosa vuol dire “passaggio per valore”

Definizione

Nel **passaggio per valore** chi chiama la funzione le consegna **una copia** del numero, non la variabile originale. La funzione lavora **sulla copia**.

Alla lavagna il titolo era: *“Funzioni con passaggio per valore: effettua una copia del parametro”*. Per capire che cosa significa davvero, dobbiamo prima guardare **come è fatta la memoria**.

2.1 Com’è fatta fisicamente la RAM

In un computer i dati del programma stanno nella **RAM**. Un **banco di RAM** è una piccola scheda (PCB) con tanti **contatti** stampati su un bordo: invece di avere i piedini come gli integrati che usiamo noi, ha delle **piste dorate** che si infilano nello slot della scheda madre. Spesso è coperto da una **placca metallica** (a volte con le lucine colorate “per farlo figo”, ma è solo estetica): la placca serve a **dissipare il calore**, perché la RAM lavora **molto velocemente** e una cosa che va veloce **scalda**, sempre.

Togliendo la placca si vedono alcuni **integrati neri** montati su un lato e sull’altro. Facciamo un conto con un banco tipico da **16 GB**: se ci sono 4 chip da una parte e 4 dall’altra, in totale **8 chip**, ciascuno contiene **2 GB** ($8 \times 2 = 16$). Un banco da 8 GB avrà 8 chip da 1 GB, uno da 32 GB 8 chip da 4 GB, oppure più chip più piccoli: dipende da come il **costruttore** decide di realizzarlo, in base a quale combinazione gli costa meno.

Questi integrati non hanno le “zampette” passanti: sono componenti **SMD** (*Surface Mount Device*, montaggio superficiale), che **non richiedono fori** e si saldano **appoggiati sopra** la scheda. Se si apre un integrato, dentro c’è un **chip di silicio** molto più piccolo del contenitore nero: il contenitore è grande solo per renderlo **maneggevole**. Il chip è collegato ai piedini esterni da **sottilissimi fili d’oro**. C’è chi prova a recuperare l’oro dai circuiti, ma da un bancale di computer si ottengono solo 2–3 grammi: non ne vale la pena.

Dentro il chip la memoria è una **grande griglia** di caselline, ciascuna capace di contenere un **1 o uno 0**. In 2 GB ci sono 2 miliardi di byte, cioè circa **16 miliardi di bit**: 16 miliardi di caselline. Potremmo immaginarle come **16 miliardi di flip-flop** (i dispositivi che “tengono memoria”, visti in elettronica); in realtà nella RAM dei computer la tecnologia è diversa: ogni bit è conservato in un minuscolo **condensatore**, un componente che **tiene una carica elettrica**. Il principio però è lo stesso: è un **elemento di memoria**.

2.2 La memoria come una cassettiera

Non ci interessa disegnare 16 miliardi di caselle: possiamo immaginare la memoria come una **cassettiera**. Pensate al **settimanale**, la cassettiera con un cassetto per ogni giorno della settimana. Ogni cassetto ha un **numero**: cassetto 0, 1, 2, 3, … fino a circa 2 miliardi. Ogni cassetto è un **byte**, diviso in **8 bit**, e possiamo scrivere ciascuno di quegli 8 bit.

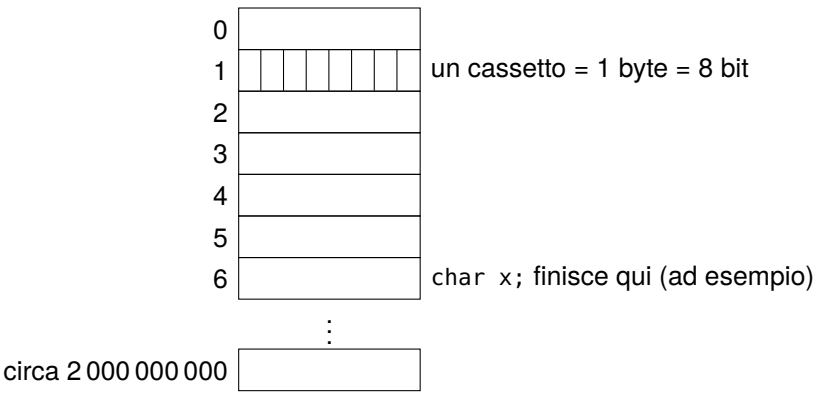


Figura 2: La memoria vista come una cassettiera numerata (dal disegno alla lavagna).

Quando nel programma **dichiariamo una variabile**, ad esempio `char x;`, la macchina **sceglie un**

cassetto libero (diciamo il numero 6) e gli mette l’**etichetta** `x`. Se riavviamo il programma, la volta dopo `x` potrebbe finire nel cassetto 422, o nel 1037: la posizione **cambia a ogni avvio**. In realtà c’è un **algoritmo** che decide, ma per noi è come se fosse **casuale**. Un `int` su Arduino occupa **2 byte** (16 bit), quindi **due cassetti** consecutivi.

Torniamo a `lampeggio(5, 3, 200)`. Il 5 scritto nella chiamata sta da qualche parte in memoria. Quando la funzione parte, il parametro `int pin` viene **creato** in un cassetto, ad esempio il 3: essendo un intero occupa 16 bit, cioè **due cassetti**, il 3 e il 4. Dentro ci viene **copiato** il valore 5. Uso apposta le parole **copiato** e **duplicato**: `pin` non è il 5 originale, è **una copia**. Per `lampeggio` questo non crea nessun problema: la funzione occupa un po’ di memoria in più, ma per noi è poca cosa.

2.3 L’esempio `duplica`: la funzione lavora su una copia

Finché la funzione si limita a *usare* il numero (come `lampeggio`, che lo usa per scegliere il piedino), il fatto che lavori su una copia non ci dà fastidio. Il problema nasce quando la funzione **modifica il dato**. Proviamo con una funzione che dovrebbe **raddoppiare** un numero:

```
void loop()
{
  int x;
  x = 5;
  duplica(x);
  // quanto vale x qui? Ancora 5!
}

void duplica(int pippo)
{
  pippo = pippo * 2;
}
```

Seguiamo che cosa succede nella cassettiera:

1. `int x;` crea `x` in un cassetto qualsiasi; `x = 5;` ci scrive dentro **5**.
2. `duplica(x)` chiama la funzione: viene creata la variabile `pippo` **in un altro cassetto** (due byte, perché è un `int`) e dentro ci viene **copiato** il 5.
3. `pippo = pippo * 2;` prende il 5 di `pippo`, lo moltiplica per 2 e scrive **10** nel cassetto di `pippo`.
4. Alla **graffa di chiusura** la funzione finisce e **“distrugge”** le sue variabili.

Quanto vale `x` alla fine? **Sempre 5**: la funzione ha lavorato **sulla copia**, non sulla `x` originale.

Nota

“Distruggere” una variabile è una parola un po’ sbagliata, che però rende l’idea. Quando una funzione finisce, **dealloca** le sue variabili: **cancella il riferimento**, cioè dimentica che `pippo` stava in quel cassetto. Il 10 **resta lì**, ma il cassetto non è più riservato: è **libero** per un’altra variabile, che prima o poi ci scriverà sopra. È come un posto a sedere riservato a un compagno finché fa la sua attività: quando ha finito, il posto torna libero, anche se lui ha lasciato lì lo zaino. Per questo, se nel `loop` chiedessimo quanto vale `pippo`, **`pippo` non esiste**: esiste solo **durante l’esecuzione** della funzione `duplica`.

2.4 Recuperare il risultato con `return`

Per farci **riconsegnare** il valore calcolato dobbiamo cambiare due cose: la funzione non è più `void` ma dichiara il **tipo del valore di ritorno** (`int`), e alla fine usa **`return`**. Nella chiamata, il risultato viene **assegnato** a una variabile:

```
void loop()
{
  int x;
  x = 5;
  x = duplica(x);
  // adesso x vale 10
}

int duplica(int pippo)
{
  pippo = pippo * 2;
  return pippo;
}
```

Ora la sequenza è: si crea `pippo` (in un cassetto ancora diverso: **ogni volta cambia**), ci si copia il 5, si sovrascrive con 10, e `return pippo` dice: “quando hai finito il calcolo, il **valore** di **tutto questo blocco** deve essere quello di `pippo`”. Il 10 viene copiato in `x`, poi `pippo` viene deallocata: ma il **valore** che ci serviva l’abbiamo **recuperato**.

Attenzione

Riassumendo: con il passaggio per valore, se la funzione **non deve dare un risultato** (è `void`) la copia non ci crea nessun problema. Se invece il dato deve essere **elaborato e riconsegnato**, bisogna usare il **valore di ritorno** (`return`) e **assegnarlo** a una variabile nella chiamata.

2.5 Un esempio più concreto: il peso di un cubo stampato in 3D

Lo **slicer** della stampante 3D, oltre a preparare il file per la stampa, ci dice **quanto filamento** serve: se gli diamo un cubo, ci dice ad esempio che servono 15–20 grammi di materiale. Come fa? Conosce il **peso specifico** del materiale (quanto pesa un centimetro cubo) e sa calcolare il **volume** dell’oggetto: **peso** = **peso specifico** $\times$ **volume**. Per un cubo, il volume è lato $\times$ lato $\times$ lato.

Per l’esempio usiamo un peso specifico di **2 grammi per centimetro cubo**. Le unità di misura tornano: $\frac{\text{g}}{\text{cm}^3} \times \text{cm}^3 = \text{g}$, cioè moltiplicando per il volume (in cm^3) il centimetro cubo “sotto” si semplifica e restano i **grammi**.

Nota

Il valore di $2\,\text{g/cm}^3$ è comodo per fare i conti a mente. Il peso specifico reale del **PLA** è circa $1{,}24\,\text{g/cm}^3$.

```
void loop()
{
  // lato del cubo in centimetri
  int lato = 5;
  int peso = calcolaPeso(lato);
}

int calcolaPeso(int x)
{
  int volume = x * x * x;
  int peso = 2 * volume;
  return peso;
}
```

Seguiamolo nella cassettiera, come alla lavagna:

1. `int lato = 5;` crea `lato` in un cassetto e ci mette **5**.
2. `int peso = calcolaPeso(lato);` crea la variabile `peso` del `loop` (alla lavagna era **in rosso**); il suo valore dipende dalla funzione, quindi è in **“stand by”** finché la funzione non ha finito.
3. Si entra in `calcolaPeso`: viene creata `x` in un altro cassetto e ci viene **copiato** il valore di `lato`, cioè 5.
4. `int volume = x * x * x;` crea `volume` e ci scrive $5 \times 5 = 25$, $25 \times 5 = \mathbf{125}$. Nel cassetto scelto c’era già un numero vecchio, ma “chi se ne frega”: viene **sovrascritto**.
5. `int peso = 2 * volume;` crea **un’altra** variabile `peso` (alla lavagna **in blu**), in un cassetto diverso da quella del `loop`: hanno lo **stesso nome** ma sono **due variabili completamente distinte**, che non hanno niente a che fare l’una con l’altra. Dentro ci va $2 \times 125 = \mathbf{250}$.
6. `return peso;` prende il valore del `peso` **blu** e lo fa diventare il valore di tutta la chiamata `calcolaPeso(lato)`: il 250 viene copiato nel `peso` **rosso**.
7. La funzione finisce e **dealloca** `x`, `volume` e il `peso` blu. I numeri restano nei cassetti, ma verranno sovrascritti da qualunque altra cosa accada nel programma. Restano `lato` (5) e il `peso` rosso, che ora vale il **risultato che ci aspettavamo**: 250 grammi.

Attenzione

Errore tipico: rifacendo il percorso, al passo 5 viene spontaneo scrivere il 250 nel `peso` **del loop**. Invece `int peso` dentro la funzione crea **una variabile nuova** (quella blu), in un altro cassetto: il `peso` rosso riceve il valore **solo con `return`**, alla fine.

Nota

Il nome del parametro (`x`) può essere **diverso** da quello della variabile passata (`lato`): la funzione riceve solo il **valore**. Allo stesso modo, una variabile creata dentro la funzione può avere lo **stesso nome** di una del `loop`: sono due oggetti con lo stesso nome ma **in posti diversi**. E se chiamassimo di nuovo la funzione, `volume` **non è detto** che finisca nello stesso cassetto di prima.

3 Il circuito del 595 su Tinkercad

Passiamo alla pratica. Il circuito è quello della dispensa **A04 di elettronica** (“Pilotare il 74HC595”): il **595** collegato a **8 LED**, ciascuno con il suo resistore. Su KiCad non andiamo ancora avanti con sbroglio e produzione perché la **macchina è in manutenzione**; intanto lo **riproduciamo su Tinkercad** e scriviamo il firmware.

Il 595 su Tinkercad si trova aprendo **tutti i componenti**: in fondo all’elenco c’è il **registro a scorrimento** (oppure si cerca “595”). Partendo dallo schema, i collegamenti fissi sono:

- piedino **8** (GND) alla **massa**;
- piedino **16** (VCC) ai **5 V** (cioè al binario rosso della breadboard, collegato ai 5 V di Arduino);
- piedino **10** (SRCLR) ai **5 V**, così il registro non viene mai cancellato;
- piedino **13** (OE) a **GND**, così le uscite sono sempre abilitate;
- le **8 uscite** ai LED, ciascuno con il suo **resistore** verso massa.

Nello schema KiCad, Arduino non è disegnato: al suo posto c’è il **connettore J1** (`Conn_01x05`, 1 fila da 5 contatti), dove andranno i fili verso Arduino. Su Tinkercad invece Arduino c’è, e i fili che nello schema arrivano a J1 vanno direttamente ai suoi piedini. I tre fili che vanno ad Arduino si possono collegare ai piedini che si vuole: non serve sceglierli in modo particolare. Nel circuito della lezione:

Arduino	piedino del 595	nome
3 (filo arancione)	14	SER (dato)
5 (filo giallo)	12	RCLK (pubblica sulle uscite)
7 (filo verde)	11	SRCLK (fa scorrere il bit)

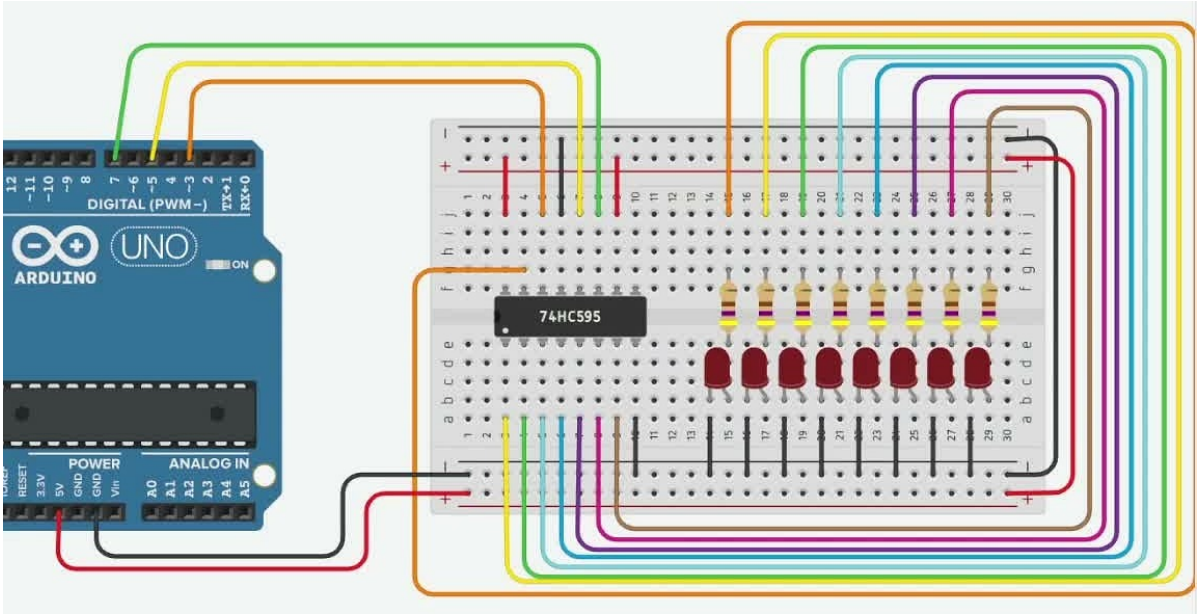


Figura 3: Il circuito finito su Tinkercad: Arduino (piedini 3, 5, 7) collegato al 74HC595, che pilota 8 LED rossi con i loro resistori. I fili sono stati fatti passare “per fuori” per evitare incroci tra il rosso e il nero e rendere il disegno più leggibile.

Attenzione

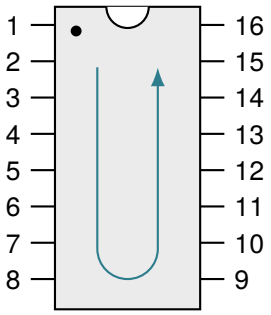
Durante il disegno due fili erano stati tirati **una casella più a sinistra** del dovuto e poi corretti. Prima di simulare, **ricontrollate ogni filo** contro lo schema: un filo spostato di un solo foro della breadboard basta perché il circuito non funzioni.

3.1 Come si numerano i piedini di un integrato

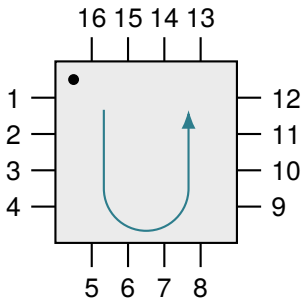
Tinkercad dà ai piedini del 595 dei nomi che **non sono gli stessi** di KiCad (ad esempio, passando con il mouse sul piedino 14 compare "ingresso"): per sapere che cos'è un piedino conviene ragionare sul suo **numero**. E il numero si ricava sempre con la stessa regola, senza bisogno del manuale.

Definizione

Si guarda l'integrato **da sopra**, con la **tacca** (la mezzaluna) in alto. Il **piedino 1** è quello **subito a sinistra della tacca** (spesso segnato anche con un pallino). Da lì si contano i piedini **in senso antiorario**: si scende lungo il lato sinistro, si risale lungo il lato destro.



DIP a 16 piedini



integrato con piedini sui 4 lati

Figura 4: La regola della numerazione: dal piedino 1, vicino alla tacca o al pallino, in senso antiorario guardando l'integrato da sopra. Vale anche per gli integrati con i piedini sui quattro lati.

Attenzione

Un errore tipico è numerare il lato destro **dall'alto verso il basso** (9, 10, 11, 12… partendo da sopra): non è coerente con la regola usata a sinistra. Contando **in senso antiorario**, dopo l'8 (in basso a sinistra) si passa al **9 in basso a destra** e si **risale** fino al 16, che sta **di fronte all'1**, dall'altra parte della tacca. È una regola vista già l'anno scorso, e vale per tutti gli integrati.

Applicando la regola al 595: il filo arancione (piedino 3 di Arduino) arriva al piedino **14**, che è **SER**; il giallo (piedino 5) arriva al **12**, che è **RCLK**; il verde (piedino 7), per esclusione, arriva all'**11**, **SRCLK**.

4 Le etichette: #define

4.1 Il problema: ricordarsi i numeri dei piedini

In `setup()` mettiamo la **configurazione**: i tre piedini usati vanno a pilotare un componente esterno, quindi sono tutti e tre **uscite**.

```
void setup()
{
  pinMode(3, OUTPUT);
  pinMode(5, OUTPUT);
  pinMode(7, OUTPUT);
}
```

Se avete scelto piedini diversi, ovviamente il codice va modificato. Ma c'è un problema: questi numeri li dovremo **riusare** in tutto il codice per decidere chi alzare e chi abbassare, e dovremo **ricordarci** ogni volta che il 3 è SER, il 5 è RCLK e il 7 è SRCLK. Con tre fili è già scomodo; immaginate un circuito con **72 fili**, cosa che in certi progetti succede davvero.

Nota

Il docente scrive il codice in **Visual Studio Code**, in un file `.cpp`, solo perché l'editor **colora** la sintassi e lo rende più leggibile; poi lo copia su Tinkercad. VS Code segna alcuni **errori** perché non conosce i comandi di Arduino: lì non contano. Voi scrivete direttamente nell'editor di codice di Tinkercad (in modalità **Testo**).

4.2 L'idea: "trova e sostituisci"

Prendiamo un testo (generato da un'intelligenza artificiale in due secondi, solo per fare l'esempio): "La luce del sole illuminava la stanza… e quella vecchia *casa* sembrava prendere vita ogni mattina", e immaginiamo che sia lungo **decine di pagine**. Se decidiamo che non era una casa ma una *capanna*, dovremmo cercare a mano ogni "casa", magari ripetuta 72 volte. Invece i programmi di videoscrittura come Word hanno il comando **trova e sostituisci**: "trova *casa*, sostituisci con *capanna*, sostituisci tutto". Potremmo anche sostituire "capanna" con il numero 72: non ha senso nel testo, ma il concetto è lo stesso. E la sostituzione si fa **prima** di consegnare il testo a chi lo deve leggere. (Altro esempio: copio il tema "cosa ho fatto quest'estate" di un compagno e con trova e sostituisci cambio il suo nome con il mio… il concetto si capisce.)

In C si può fare esattamente questo con **`#define`**: si crea un'**etichetta** (*label*) e ogni volta che il compilatore la trova la **sostituisce** con quello che abbiamo scritto, **prima di compilare**.

```
#define SER 3
#define RCLK 5
#define SRCLK 7

void setup()
{
  pinMode(SER, OUTPUT);
  pinMode(RCLK, OUTPUT);
  pinMode(SRCLK, OUTPUT);
}
```

Da ora in poi non parliamo più del piedino 3, 5 o 7, ma dei piedini **SER**, **RCLK** e **SRCLK**: le conversioni le fa la macchina quando compila il programma e lo manda ad Arduino. Se siamo alla riga 6522 di un programma lungo 8000 righe e dobbiamo accendere il filo collegato a SER, è molto più facile ricordarsi **il nome** che il numero. A che piedino è collegato? È scritto una volta sola, **in cima al file**.

Nota

La sostituzione può essere fatta **con qualunque cosa**, anche con un'istruzione intera: ad esempio `#define ACCENDI_SER digitalWrite(SER, HIGH)`, e poi scrivere solo `ACCENDI_SER` nel codice. Portando l'idea all'estremo potremmo costruirci un nostro linguaggio di programmazione fatto di `#define`, ma **non si fa mai**: a noi basta dare un **nome a ogni filo**.

4.3 `#define` non è una variabile

Attenzione

Un `#define` **assomiglia** a una variabile (un nome con dentro un numero), ma è una cosa ben diversa:

- una **variabile** è un nome che **occupa spazio in memoria** (un cassetto) e che posso **cambiare quando voglio** mentre il programma gira;
- un `#define` è una **costante**: un'etichetta sostituita **prima di compilare**, che **non occupa memoria** e che **non si può modificare** durante l'esecuzione (a *runtime*). Posso cambiarla solo modificando la riga in cima al file, e poi devo **ricompilare**.

In C c'è una **regola non scritta**, che quasi tutti i programmatori seguono: le **variabili** si scrivono in **minuscolo**, le **costanti** in **MAIUSCOLO**. Se nessuno la rispetta il programma funziona lo stesso, ma così chi legge un nome tutto maiuscolo sa subito, anche senza sapere dove è stato definito, che quel valore **non si può toccare**.

5 Come si carica un dato nel 595

5.1 Ripasso: i tre segnali

Il pilotaggio del 595 è spiegato nella dispensa A04 di elettronica; ricordiamo i tre segnali:

- **SER**: l'**ingresso dei dati**, dove presentiamo un bit alla volta;
- **SRCLK**: il **clock** che fa **entrare** il bit presentato su SER nel registro a scorrimento, spostando di un posto quelli già dentro;
- **RCLK**: il segnale che **pubblica** sulle uscite, **in un colpo solo**, tutto quello che abbiamo caricato, così **non si vedono scorrere** i bit dentro il registro.

Ricordate il **tubo con le palline**: il tubo ha **otto finestrelle**, e noi infiliamo dentro palline **rosse** (LED acceso, 1) o **bianche** (LED spento, 0). Per infilare una pallina prima la **presentiamo** davanti all'imboccatura scegliendo il colore (SER alto o basso), poi la **spingiamo dentro** (un colpo di SRCLK). Mentre riempiamo il tubo, le finestrelle sono **coperte**: le scopriamo ("scattiamo la fotografia" con RCLK) solo quando tutte le palline sono al loro posto.

5.2 L'esempio: la sequenza da caricare

Scegliamo quali degli 8 LED accendere. Nell'esempio della lezione, partendo dalla **prima pallina da inserire**: rossa, bianca, rossa, bianca, rossa, rossa, bianca, rossa, cioè `1 0 1 0 1 1 0 1`.

Attenzione

Il 595 è un **registro a scorrimento**: la **prima pallina** che infiliamo è quella che, alla fine, si ritrova **in fondo** al tubo. Quindi bisogna ragionare **al contrario**: se si parte dal LED sbagliato, la sequenza si vede **girata**. Niente di grave: basta **ruotare la disposizione dei LED** (o leggere la fila dall'altra parte) per sistemarla.

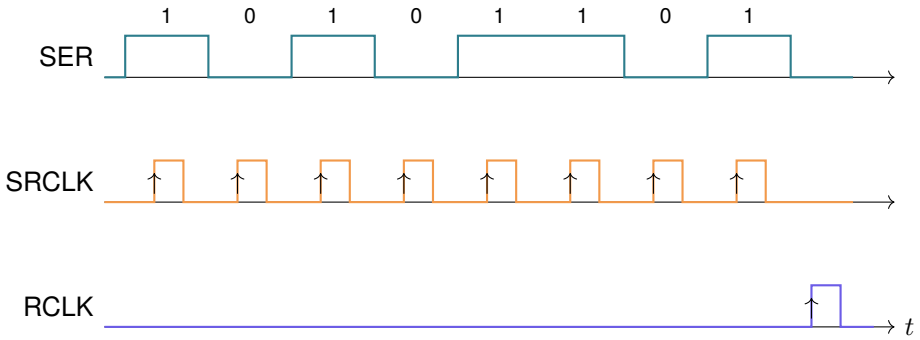


Figura 5: Il cronogramma disegnato alla lavagna: su SER si presentano uno dopo l'altro gli 8 bit `1 0 1 0 1 1 0 1`; a ogni bit corrisponde un impulso di SRCLK (il bit entra sul fronte di salita); alla fine un solo impulso di RCLK pubblica il risultato sulle uscite. Quando abbiamo finito di caricare, RCLK riceve il suo colpo di clock e solo allora i LED cambiano.

5.3 Prima versione: una pallina alla volta

Per infilare la prima pallina (rossa) scriviamo:

```
// presento la pallina ROSSA davanti al tubo
digitalWrite(SER, HIGH);
delay(1);
// la spingo dentro: alzo e abbasso SRCLK
digitalWrite(SRCLK, HIGH);
delay(1);
digitalWrite(SRCLK, LOW);
delay(1);
```

"Metti il dato, poi alza e abbassa": come **premere il pulsante** per scattare la foto e poi **togliere il dito**. Se scrivessimo solo le tre istruzioni `digitalWrite` una dopo l'altra, Arduino le eseguirebbe in un tempo così breve che il 595 potrebbe **non fare in tempo** a capire che cosa è successo. Per sicurezza mettiamo **uno spazietto** tra un'operazione e l'altra: un `delay`. Ne basta pochissimo: **1 millisecondo**. Il 595 è abbastanza veloce da ricevere anche mille dati al secondo, e in realtà si potrebbe andare **molto più veloce**; con i nostri `delay(1)` (tre per ogni bit) carichiamo qualche centinaio di bit al secondo, più che sufficiente. Se si vuole **vedere** l'operazione con calma, si può mettere anche `delay(100)`.

Per la seconda pallina (bianca) si ripetono le stesse righe con `LOW` su SER, e così via per tutte e otto. È lunghissimo: stiamo scrivendo righe e righe per una cosa che dovrebbe essere semplice.

5.4 Seconda versione: la funzione `carica()`

Notiamo che c'è un pezzo che **ripetiamo ogni volta**, identico: alza SRCLK, aspetta, abbassa SRCLK, aspetta. Abbiamo parlato delle macro: raggruppiamolo in una **funzione**. Non deve riconsegnare niente, quindi è **`void`**; la chiamiamo `carica` (si potrebbe chiamare anche `clock`, perché è più quello che fa).

```
void carica()
{
  digitalWrite(SRCLK, HIGH);
  delay(1);
  digitalWrite(SRCLK, LOW);
  delay(1);
}
```

Ora ogni pallina richiede solo tre righe invece di sei: abbiamo **dimezzato** il codice, e quelle quattro righe le abbiamo scritte **una volta sola**, nella funzione. Alla fine degli 8 bit manca ancora un'operazione, diversa da tutte le altre: il colpo di **RCLK**.

```
#define SER 3
#define RCLK 5
#define SRCLK 7

void setup()
{
  pinMode(SER, OUTPUT);
  pinMode(RCLK, OUTPUT);
  pinMode(SRCLK, OUTPUT);
}

void loop()
{
  // inserisco la prima pallina ROSSA
  digitalWrite(SER, HIGH);
  delay(1);
  carica();

  // inserisco la seconda pallina BIANCA
  digitalWrite(SER, LOW);
  delay(1);
  carica();

  // inserisco la terza pallina ROSSA
  digitalWrite(SER, HIGH);
  delay(1);
  carica();

  // inserisco la quarta pallina BIANCA
  digitalWrite(SER, LOW);
  delay(1);
  carica();

  // inserisco la quinta pallina ROSSA
  digitalWrite(SER, HIGH);
  delay(1);
  carica();

  // inserisco la sesta pallina ROSSA
  digitalWrite(SER, HIGH);
  delay(1);
  carica();

  // inserisco la settima pallina BIANCA
  digitalWrite(SER, LOW);
  delay(1);
  carica();

  // inserisco l'ottava pallina ROSSA
  digitalWrite(SER, HIGH);
  delay(1);
  carica();

  // attivo RCLK per trasferire i dati dal registro a uscita
  digitalWrite(RCLK, HIGH);
  delay(1);
  digitalWrite(RCLK, LOW);
  delay(1);
}

void carica()
{
  digitalWrite(SRCLK, HIGH);
  delay(1);
  digitalWrite(SRCLK, LOW);
  delay(1);
}
```

Nota

Ogni pallina ha un `delay(1)` prima di `carica()`: si potrebbe toglierlo da lì e metterne uno in più **dentro** `carica()`, all'inizio. Funzionerebbe uguale, e anzi sarebbe la scelta migliore; per ora lo lasciamo fuori perché il discorso prosegue.

Nota

In Arduino (e su Tinkercad) una funzione come `carica()` si può scrivere **sotto** il `loop()` che la usa: l'ambiente di Arduino aggiunge da solo la dichiarazione. In un programma C "normale" andrebbe dichiarata prima.

5.5 I LED "esplodono": attenzione alla corrente

Copiando il codice su Tinkercad e avviando la simulazione, i LED **esplodono**. Il motivo è la **corrente**: con resistori da 220 Ω ogni LED assorbe più di 10 mA, e con 8 LED accesi insieme la corrente totale che passa nel 595 supera quella che il chip è in grado di sopportare. La soluzione è **aumentare i resistori**, ad esempio a **470 Ω**: la corrente di ogni LED si **dimezza** circa, e il circuito per quello che interessa a noi funziona lo stesso.

Esempio

Il conto. In aula abbiamo fatto una **stima a voce**: circa 20 mA per LED, quindi circa 160 mA in tutto, troppi per il chip; con 470 Ω la corrente circa si dimezza. Ecco il conto più preciso. Con un LED rosso (circa 2 V ai suoi capi) e 5 V di alimentazione, sul resistore restano circa 3 V.

- Con 220 Ω: $I = 3\,\text{V}/220\,\Omega \approx 13{,}6\,\text{mA}$ per LED, quindi con 8 LED accesi circa **109 mA**.
- Con 470 Ω: $I = 3\,\text{V}/470\,\Omega \approx 6{,}4\,\text{mA}$ per LED, quindi con 8 LED circa **51 mA**.

Il datasheet del 74HC595 indica come limite massimo **70 mA** in totale attraverso i piedini di alimentazione (VCC o GND) e 35 mA per ogni singola uscita: con 220 Ω e tutti i LED accesi si va oltre, con 470 Ω si resta sotto.

Dopo la correzione la sequenza appare, ma **al contrario** rispetto a come l'avevamo pensata: è l'effetto del registro a scorrimento (la prima pallina finisce in fondo), e basta leggere la fila dall'altro lato.

6 Terza versione: un numero e l'AND bit a bit

6.1 Il numero 181

Il codice è ancora lungo. Notiamo che i blocchi delle palline sono **tutti uguali**, al netto di `HIGH` o `LOW`. Ricordiamo che **`HIGH`** numericamente è **1** e **`LOW`** è **0**. Se trovassimo una **formula** che calcola l'1 o lo 0 partendo da un numero, potremmo scrivere tutti i blocchi allo stesso modo.

Prendiamo la sequenza di LED e scriviamola come **numero binario**. Numeriamo le posizioni da **0 a 7** partendo da destra: la pallina inserita per prima è il **bit 0**, l'ultima il **bit 7**. Leggendo dal bit 7 al bit 0 la sequenza diventa `10110101`. Per convertirla in **decimale** si dà un **peso** a ogni cifra, $2^0, 2^1, \ldots, 2^7$, e si sommano solo i pesi delle cifre che valgono 1:

posizione	7	6	5	4	3	2	1	0
peso	128	64	32	16	8	4	2	1
bit	1	0	1	1	0	1	0	1
si somma	128	–	32	16	–	4	–	1

$128 + 32 + 16 + 4 + 1 = 181$. La nostra sequenza di LED è il numero **181 scritto in binario**.

6.2 Per il computer, ogni numero diverso da zero è "vero"

Definizione

Per il computer **solo lo zero è falso** (basso, spento, `LOW`). **Qualunque altro numero** è **vero**: alto, acceso, `HIGH`.

Quindi `digitalWrite(3, 72)` **accende** il piedino 3, perché 72 è diverso da zero; anche `digitalWrite(3, -9)` lo accende. L'unico modo per spegnerlo è scrivere **0**. Ci serve allora un'operazione che, a partire dal 181, dia **zero** quando il bit che ci interessa è 0, e **un numero diverso da zero** quando è 1.

6.3 L'AND bit a bit (`&`)

L'operatore **AND** dà **1 solo se entrambi** gli ingressi sono 1. L'**AND bit a bit** (in C **`&`**, una sola e commerciale) lo applica a **ogni coppia di bit** di due numeri: in pratica è come **moltiplicare** bit per bit. È un operatore visto l'anno scorso.

Facciamo l'AND tra 181 e un numero che ha **un solo 1** nella posizione che vogliamo leggere:

	`10110101` (181)			`10110101` (181)	
`&`	`00000001`		`&`	`00000010`	
=	`00000001` $\neq 0$: **acceso**		=	`00000000` $= 0$: **spento**	

Nel primo caso il risultato è diverso da zero: il LED si accende, ed è proprio quello che vogliamo (il bit 0 di 181 è 1). Nel secondo caso il risultato è zero: il LED resta spento, ed è proprio quello che vogliamo (il bit 1 di 181 è 0). Per ogni pallina, quindi, facciamo l'AND tra 181 e un numero con **un solo 1** che ogni volta **cambia posizione**: prima pallina prima posizione, seconda pallina seconda posizione, e così via fino all'ottava.

In Arduino un numero binario si può scrivere con la **B** davanti, ad esempio `B00000001`: sono **8 cifre**, sette zeri e un 1.

```
// inserisco la prima pallina ROSSA
digitalWrite(SER, 181 & B00000001);
delay(1);
carica();

// inserisco la seconda pallina BIANCA
digitalWrite(SER, 181 & B00000010);
delay(1);
carica();

// inserisco la terza pallina ROSSA
digitalWrite(SER, 181 & B00000100);
delay(1);
carica();
```

Abbiamo scritto 181 per semplicità: è **la stessa cosa** di `B10110101`, l'abbiamo appena calcolato.

Nota

La notazione `B00000001` è una comodità di **Arduino**. In C++ (e nelle versioni recenti del C) un numero binario si scrive con il prefisso `0b`, ad esempio `0b00000001`, e anche Arduino lo accetta.

6.4 Lo scorrimento a sinistra (<<)

Ora ogni blocco differisce dagli altri **solo per la posizione dell'1**. C'è un modo per scrivere `B00000010` partendo da `B00000001`? Sì: **spostarlo a sinistra** di una posizione. L'operatore **<<** (doppio minore), visto anche questo l'anno scorso, esegue una **traslazione a sinistra** del byte:

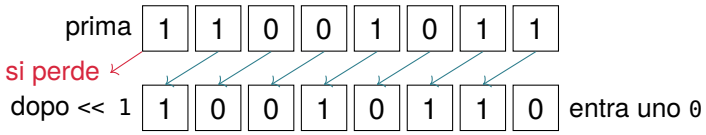


Figura 6: Traslazione a sinistra di una posizione: ogni bit si sposta di un posto a sinistra, il bit più a sinistra si perde e da destra entra sempre uno 0. Lo scorrimento a destra (>>) fa lo stesso nel verso opposto.

Quindi:

- `B00000010` è `(B00000001 << 1);`
- `B00000100` è `(B00000001 << 2);` e così via con 3, 4, 5, 6, 7;
- anche il primo si può scrivere con lo stesso schema: `(B00000001 << 0)`, cioè spostato di zero posizioni.

```
// inserisco la prima pallina ROSSA
digitalWrite(SER, 181 & (B00000001 << 0));
delay(1);
carica();

// inserisco la seconda pallina BIANCA
digitalWrite(SER, 181 & (B00000001 << 1));
delay(1);
carica();

// ... e cosi' via fino all'ottava
digitalWrite(SER, 181 & (B00000001 << 7));
delay(1);
carica();
```

Attenzione

Fate attenzione all'**autocompletamento** dell'editor: copiando i blocchi è facile ritrovarsi con un numero di partenza sbagliato. In aula è successo proprio così: l'editor aveva completato `181 & (B00010000 << 4)` e `181 & (B00100000 << 5)`, cioè un byte **già spostato** che poi veniva spostato **di nuovo**, invece di `B00000001 << 4` e `B00000001 << 5`. Il numero da spostare deve essere **sempre** `B00000001`: cambia solo **di quanto** lo spostiamo. Con un errore così la sequenza dei LED risulta diversa da quella attesa.

7 Versione finale: il ciclo `for`

Adesso arriva il bello. I blocchi delle otto palline sono **tutti uguali** al netto di **un numeretto**, quello dello spostamento, che vale 0, poi 1, 2, 3, ... fino a 7. Un pezzo di codice che si **ripete 8 volte**, e ogni volta un numero aumenta di 1: è esattamente il lavoro di un **ciclo `for`**. Al posto del numeretto mettiamo la variabile **`i`**, che la prima volta vale 0, poi 1, poi 2, e così via. Tutti gli otto blocchi si buttano via e restano poche righe:

```
#define SER 3
#define RCLK 5
#define SRCLK 7

void setup()
{
  pinMode(SER, OUTPUT);
  pinMode(RCLK, OUTPUT);
  pinMode(SRCLK, OUTPUT);
}

void loop()
{
  for (int i = 0; i < 8; i++)
  {
    digitalWrite(SER, 181 & (B00000001 << i));
    delay(1);
    carica();
  }

  // attivo RCLK per trasferire i dati dal registro a uscita
  digitalWrite(RCLK, HIGH);
  delay(1);
  digitalWrite(RCLK, LOW);
  delay(1);
}

void carica()
{
  digitalWrite(SRCLK, HIGH);
  delay(1);
  digitalWrite(SRCLK, LOW);
  delay(1);
}
```

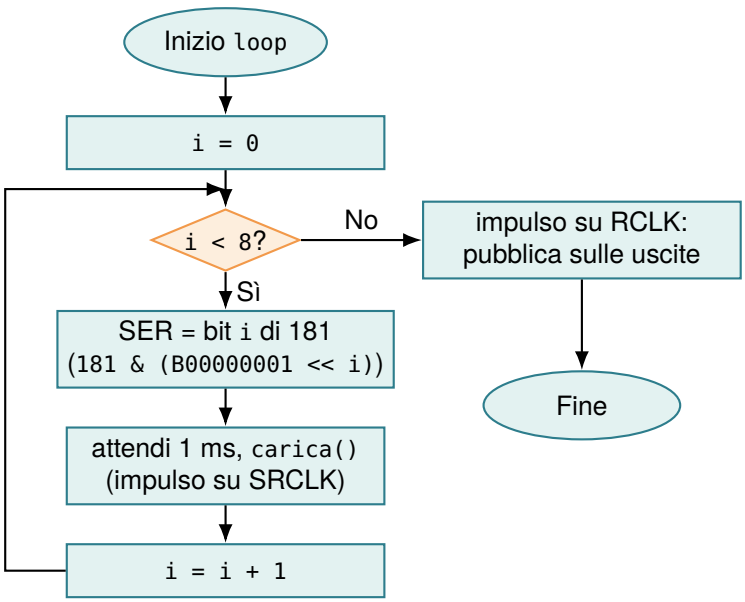


Figura 7: Diagramma di flusso di una passata del `loop()` nella versione finale. Il `loop()` poi ricomincia da capo, all'infinito: il "Fine" indica la fine di una passata.

Il codice era lungo circa **70 righe**, ora è circa **35**, e funziona allo stesso modo. Qual è meglio? Questa versione: più è **corto** il codice, meno probabilità ci sono di **sbagliare** (è una regola di vita dei programmatori). E c'è un altro vantaggio: per mostrare un'altra combinazione di LED basta **cambiare il numero 181**.

Attenzione

Errore tipico: se si accendono (o si spengono) **tutti i LED**, probabilmente nel `for` è rimasto `<< 0` invece di `<< i`. In quel caso ogni volta viene buttato fuori **sempre lo stesso bit** (il bit 0 di 181, che vale 1), mentre il bit deve **cambiare in funzione della posizione**. Altri errori tipici: **fili non collegati** davvero ai fori della breadboard, e **righe perse** copiando il codice con Ctrl+C / Ctrl+V. Controllate sempre bene che cosa avete scritto: se il codice funziona sul computer del docente, deve funzionare uguale anche sul vostro.

8 Cosa ricordare di questa lezione

- Nel **passaggio per valore** la funzione riceve **una copia** del parametro: le modifiche fatte dentro la funzione **non cambiano** la variabile originale.
- La memoria è come una **cassettiera** numerata: ogni variabile occupa uno o più cassetti (`char` 1 byte, `int` 2 byte su Arduino), scelti dalla macchina in modo per noi **casuale**.
- Alla fine di una funzione le sue variabili vengono **deallocate**: si cancella il riferimento, il valore resta ma il cassetto torna libero. Per riavere un risultato si usa **`return`** e lo si **assegna**: `peso = calcolaPeso(lato);`.
- Due variabili con lo **stesso nome**, una nel `loop` e una nella funzione, sono **due variabili distinte**.
- I piedini di un integrato si numerano guardandolo **da sopra**: il **piedino 1** è a sinistra della **tacca**, poi si procede in **senso antiorario**. Nel 595: 14 SER, 12 RCLK, 11 SRCLK, 10 SRCLR (a 5 V), 13 OE (a GND), 8 GND, 16 VCC.
- **`#define`** crea un'**etichetta** sostituita **prima di compilare** (come "trova e sostituisci"): è una **costante**, non occupa memoria e non si può cambiare a runtime. Le costanti si scrivono in **MAIUSCOLO**, le variabili in minuscolo.
- Per caricare un bit nel 595: si presenta il bit su **SER**, si dà un impulso su **SRCLK** (la funzione `carica()`); dopo 8 bit, un impulso su **RCLK** pubblica il risultato. Il primo bit inserito finisce **in fondo**.
- Con 8 LED accesi insieme attenzione alla **corrente totale** nel 595: con 470 Ω invece di 220 Ω si resta sotto il limite.
- Per il computer **solo lo zero è falso**. `181 & (B00000001 << i)` vale zero se il bit `i` di 181 è 0, e un numero diverso da zero se è 1: dentro un **`for`** da 0 a 7 carica tutti gli 8 bit.

Esempio

Per la verifica del 29 ottobre: la verifica copre **tutto il contenuto delle dispense**, **compresa questa**, e il **firmware dell'anno scorso** (dispense A01 e A01bis).